University of Glasgow | School of Computing Science

Honours Individual Project Dissertation

Trust Aware Stock Trend Prediction and Investment Strategy Optimisation Using Deep Learning

Dylan ShengHong Huang

March 26, 2026

Abstract

This dissertation addresses the gap in stock market prediction between accuracy-based metrics and economic value by integrating multi-modal predictions, probability calibration and principled capital allocation into a single end-to-end framework, evaluating whether diverse predictive signals fed into multiple deep-learning architectures (LSTM & GRU variants) trained to predict stock direction and magnitude can yield robust and trustworthy forecasts implemented with risk-aware capital allocation strategies.

The framework incorporates prior model predictions and their historical errors as meta-features, enabling the system to learn when its outputs should be trusted. In addition, a variable-horizon forecasting approach is employed to move beyond traditional fixed-horizon models, allowing the system to capture both short-term fluctuations and long-term trends.

Results show that this approach improves decision stability and reduces excessive turnover compared to uncalibrated baselines, furthermore, risk-aware capital allocation strategies consistently outperform naïve staking strategies, with the hybrid Kelly-Modern Portfolio Theory achieving the strongest risk-adjusted return of TODO: XX% over 100 trading days, TODO: X times higher than the previous baseline set over the same time-frame. These findings lay the groundwork for future research into trust-aware financial prediction systems that prioritises both accuracy and economic value.

Acknowledgements

I would like to express my thanks to Enrique Alejandro González Amador, a former MSc student at the University of Glasgow School of Computing Science, whose dissertation on Adapting Natural Language Processing and Deep Learning Tools to an end-to-end Day Trading System (2025) and, of course, to Prof. Jose Joemon who not only supervised Enrique but myself as well. Without his experience and guidance this project could not have reached the level that it has.

Education Use Consent

I hereby grant my permission for this project to be stored, distributed and shown to other University of Glasgow students and staff for educational purposes. **Please note that you are under no obligation to sign this declaration, but doing so would help future students.**

Signature: Dylan ShengHong Huang Date: 26 March 2026

Contents

1 Introduction

1.1 Motivation

Financial markets are inherently noisy, non-stationary systems in which stock prices are influenced by complex interactions between historical price dynamics and rapidly evolving external information. Despite decades of research, consistently forecasting market behaviour remains challenging, not only due to stochastic price movements but also due to the relationship between predictive signals and future returns varying across time and market regimes. As a result, even small modelling assumptions can significantly impact downstream decision-making.

Recent advances in deep learning have led to measurable improvements in stock movement prediction, particularly through the use of sequence models such as Long Short-Term Memory(LSTM) and Gated Recurrent Unit (GRU) networks, as well as the incorporation of unstructured textual data such as news and social media. However, much of the existing literature evaluates predictive success primarily through accuracy-based metrics, often without sufficient consideration of how model outputs are subsequently used in financial decision-making. In practice, predictive accuracy alone is insufficient, investment decisions depend critically on the reliability of predicted probabilities and the ability to translate forecasts into appropriately sized positions under risk.

In practical investment settings, model outputs are rarely consumed as hard class labels, instead, they are interpreted as probabilities that inform position sizing and risk management. This places importance on both the directional accuracy and the reliability of the prediction. A well-calibrated model produces probabilities estimates whose empirical frequencies align with observed outcomes in practice, whilst poorly calibrated models generate unreliable probabilities that are treated as meaningful inputs to allocation rules leading to over- or under-sized positions, unstable trading behaviour and amplified drawdowns, even when headline accuracy metrics appear strong.

Beyond calibration, a further limitation of many existing approaches is the implicit assumption that a model's predictive reliability is static. In reality, model performance fluctuates over time due to regime shifts, changes in market microstructure and evolving information flows. Models rarely account for their own historical errors when making new predictions and thus may continue to express high confidence even during periods of deteriorating performance. Incorporating information about prior predictions and their realised errors offers a principled way for a system to assess its own reliability and adapt its behaviour accordingly.

Finally, prediction and capital allocation are often treated as separate problems, with models evaluated independently of the investment strategies they enable. This separation obscures the true economic value of predictive improvements and limits the interpretability of reported results Investment performance depends jointly on the quality of predictive signals, the reliability of associated probabilities and the manner in which capital is allocated under uncertainty.

This project is motivated by the need for an integrated framework that unifies multimodal prediction, model self-assessment, probability calibration and principled capital allocation within a single coherent system. By combining heterogeneous data sources with information derived from prior predictions and their realised errors, the framework seeks to improve both the reliability of forecasts and their practical utility under non-stationary market conditions. Through the joint

evaluation of statistical performance and economic outcomes, this work aims to bridge the gap between predictive modelling and real-world investment decision-making, demonstrating how certainty-aware learning and risk-sensitive allocation can support more stable and economically meaningful trading strategies in dynamic environments.

1.2 Purpose

The purpose of this project is to design and evaluate an end-to-end, multi-horizon trading system that integrates multimodal predictive signals, probability calibration and risk-aware capital allocation within a single coherent pipeline. Rather than optimising predictive accuracy in isolation, the system is designed to assess the reliability and economic viability of model-driven investment decisions under realistic market conditions.

This work builds upon *Stock Movement Prediction from Tweets and Historical Prices* by **?**, which is used as a baseline for comparison. However, the focus of this project extends beyond next-day prediction and single-model evaluation by exploring variable forecasting horizons, multimodal feature integration and multiple predictive model formulations. Crucially, the value of these predictive signals is assessed through their interaction with capital allocation strategies in a simulated trading environment, rather than through benchmarking metrics alone. To account for non-stationarity and regime shifts, the proposed system incorporates prior predictions and their realised errors as meta-features, enabling the model to capture time-varying predictive reliability.

To this end, the dissertation evaluates multiple deep-learning architectures and systematically examines their integration with several capital allocation strategies, including full Kelly, fractional Kelly, mean–variance portfolio theory (MPT) and a hybrid fractional Kelly-MPT approach. By jointly analysing predictive performance, calibration quality and financial outcomes, this project aims to bridge the gap between statistical forecasting and practical investment decision-making.

1.2.1 Contributions

The following key contributions are made:

- **Stock direction and magnitude prediction formulations**, including binary classification for directional movement, regression for return magnitude and an ordinal classification approach that discretises returns into buckets to estimate expected return.
- **An end-to-end multimodal prediction and investment framework** that integrates historical price data, textual signals and auxiliary features within a single automated pipeline, designed for realistic walk-forward evaluation.
- **A variable-horizon trading system with an early-exit mechanism**, enabling positions to be held for multiple days while allowing premature exit when predicted downside risk increases, thereby improving flexibility relative to fixed-horizon strategies.
- **The explicit incorporation of model self-assessment through meta-features**, using prior predictions and their realised errors to capture time-varying predictive reliability in non-stationary financial markets.
- **A systematic application of probability calibration to deep-learning models**, enabling model confidence scores to be interpreted as meaningful probabilities suitable for downstream risk-sensitive decision-making.
- **A comparative evaluation of capital allocation strategies**, including full Kelly, fractional Kelly, mean-variance portfolio theory (MPT) and a hybrid fractional Kelly-MPT approach, assessed using economic performance measures.

- **An evaluation methodology grounded in economic outcomes**, assessing return, drawdown and risk-adjusted performance alongside traditional accuracy-based predictive metrics.

1.2.2 Novelty

The novelty of this work lies not in proposing new model architectures, but in the integrated treatment of multimodal prediction, uncertainty estimation, model reliability and capital allocation. By embedding these components within a unified framework and evaluating them through economic outcomes, this dissertation advances existing stock prediction research towards more trustworthy and practically deployable system.

1.3 Report Structure

This dissertation is organised into eight chapters.

Chapter 2, *Background and Related Work*, reviews prior research in stock movement prediction and outlines the key theoretical foundations of the project, including LSTM and GRU models, bidirectional variants, ordinal regression, BERT, probability calibration methods (isotonic regression and temperature scaling), the Kelly Criterion, mean-variance portfolio theory (MPT) and the use of meta-features.

Chapter 3, *Analysis and Research Questions*, formalises the problem setting, defines the prediction tasks, discusses constraints such as temporal integrity and non-stationarity as well as presenting the research questions guiding the evaluation.

Chapter 4, *System Design*, describes the architecture of the end-to-end trading framework, including multimodal data integration, model formulations for direction and magnitude prediction, meta-feature incorporation, calibration strategy and the investment simulation design.

Chapter 5, *Implementation*, outlines the practical realisation of the system, covering data preprocessing, feature engineering, model training, calibration, and capital allocation within a walk-forward simulation environment.

Chapter 6, *Evaluation and Results*, reports predictive performance, calibration quality, and economic outcomes such as return, drawdown, and risk-adjusted measures, alongside comparative analyses of model and allocation strategies.

Chapter 7, *Discussion*, interprets the findings in relation to the research questions and reflects on the interaction between multimodal prediction, model reliability, calibration, and capital allocation.

Chapter 8, *Conclusion and Future Work*, summarises the main contributions, discusses limitations, and outlines directions for future research.

Additional material, including extended results and implementation details, is provided in the Appendices.

2 Background & Related Work

2.1 Stock Movement Prediction

Stock movement prediction (SMP) aims to forecast future price direction or return magnitude using historical market data and auxiliary information. A fundamental challenge in this domain is the time-varying and regime-dependent nature of financial markets. Early empirical evidence by **?** demonstrated that stock market volatility is persistent, clustered and strongly associated with macroeconomic uncertainty, recessions, and financial crises. Crucially, volatility dynamics are not constant over time, and no single macroeconomic factor fully explains their variation. This work established that financial markets are inherently non-stationary, with structural shifts in risk conditions occurring over time.

The implication for prediction models is clear in that the assumptions of stable statistical relationships are often violated in practice. Forecasting systems must therefore operate under changing volatility regimes and evolving market conditions, motivating adaptive and sequence-aware modelling approaches.

2.1.1 From Statistical Models to Deep Learning

Early approaches to SMP relied on classical statistical models such as ARIMA, which assume linear relationships and weak stationarity. While these models can capture short-term autocorrelation patterns, their performance is limited in the presence of non-linearity and structural breaks (**?**).

With the emergence of machine learning and deep learning, recurrent architectures such as LSTM and GRU networks became widely adopted for financial time-series modelling. These models are capable of capturing long-term dependencies and non-linear interactions within sequential data. Large-scale empirical evidence by **?** shows that LSTM-based models trained on S&P 500 constituents (1992–2015) outperform classical baselines such as ARIMA, Random Forests, and shallow neural networks when evaluation respects temporal ordering. However, performance was shown to decay post-2010, highlighting regime sensitivity and the fragility of learned patterns.

These findings suggest that while deep sequence models can extract predictive structure, their effectiveness depends heavily on market conditions and proper evaluation protocols.

2.1.2 Multimodal Extensions

Beyond price-only models, researchers have incorporated textual information such as news headlines and social media posts to capture sentiment and narrative signals not immediately reflected in historical prices. **?** introduced *StockNet*, modelling market uncertainty using a recurrent variational autoencoder combined with an auxiliary temporal prediction head. Evaluated on Twitter and price data across 88 U.S. stocks (2014–2016), the model outperformed ARIMA, Random Forest, and text-only baselines. However, the use of dead-zone filtering, which excludes small-movement days, may have inflated reported accuracy by simplifying the classification task.

Building upon this direction, ? proposed a Multimodal Attention Network (MMAN) that integrates credibility-aware textual features with price encoders through intra- and inter-modal attention mechanisms. Ablation studies demonstrate that both credibility modelling and cross-modal attention contribute positively to performance, suggesting that structured multimodal fusion can enhance predictive capability.

In parallel, robustness to market noise has been explored through adversarial training. ? introduced an adversarially trained LSTM (Adv-LSTM), incorporating worst-case perturbations during training to improve generalisation under stochastic conditions. Reported improvements over prior baselines indicate enhanced resilience to noisy inputs.

More recent work has investigated multi-scale modelling and extended temporal windows, demonstrating that incorporating longer historical context can further improve statistical performance. Nevertheless, across these multimodal approaches, evaluation remains predominantly focused on classification metrics such as accuracy and MCC. Label filtering and the absence of cost-aware economic assessment are recurring limitations, leaving open the question of whether statistical improvements translate into meaningful financial performance.

2.1.3 Limitations in Existing Literature

Across these studies, several patterns emerge:

- Deep sequence models generally outperform classical statistical baselines under proper temporal evaluation.
- Multimodal fusion of textual and price data often yields incremental predictive gains.
- Robustness mechanisms (e.g., adversarial training, multi-scale modelling) improve statistical performance.

However, many studies:

- Evaluate performance primarily using accuracy or correlation-based metrics.
- Apply filtering techniques that simplify the prediction task.
- Do not assess economic value under realistic trading costs.
- Assume static model reliability despite regime-dependent volatility.

These limitations motivate a more integrated approach that jointly considers predictive modelling, uncertainty estimation, model reliability, and economic performance within a unified evaluation framework.

2.2 Deep Learning for Time-Series Modelling

Early recurrent neural networks (RNNs) were theoretically well-suited for sequence modelling but struggled in practice due to the vanishing and exploding gradient problem. As demonstrated by ?, gradients propagated through time either decay or grow exponentially as sequence length increases, preventing effective learning of long-term dependencies. This limitation makes standard RNNs unreliable for tasks requiring memory over extended time horizons.

To address this, ? introduced the Long Short-Term Memory (LSTM) architecture. LSTMs incorporate gated memory cells designed to preserve stable gradient flow across long temporal spans. By regulating information flow through input, forget and output gates, LSTMs enable controlled memory retention and selective forgetting. This architecture allows the network to learn dependencies spanning hundreds or even thousands of time steps, overcoming the instability inherent in traditional RNN training.

2.2.1 Long Short-Term Memory (LSTM)

Subsequent analyses, including ?, clarify that the strength of LSTMs lies in their gated structure and explicit memory cell. The separation between hidden state and cell state permits stable error propagation while protecting memory from irrelevant noise. This design enables LSTMs to model long-term temporal dependencies more reliably than earlier recurrent architectures.

In time-series domains such as finance, where regime persistence and volatility clustering occur, the ability to retain information over extended periods is particularly relevant. However, LSTMs are not without limitations. Model capacity is finite, architectural design choices significantly influence performance, and increasing network size does not automatically guarantee improved long-term reasoning. TODO: LSTM DIAGRAM

2.2.2 Gated Recurrent Units (GRU)

The Gated Recurrent Unit (GRU) was introduced as a simplified variant of the LSTM, merging the input and forget gates into a single update gate and removing the explicit memory cell. This reduces architectural complexity and parameter count while retaining gating mechanisms that regulate temporal information flow.

Empirical comparisons, such as those conducted by ?, suggest that GRUs often achieve performance comparable to LSTMs on moderately complex sequence tasks while offering improved computational efficiency. However, for highly complex or long-range dependency structures, LSTMs may demonstrate superior stability and representational capacity.

In financial time-series modelling, this trade-off implies that GRUs may suffice for short-term pattern extraction, whereas LSTMs may better capture longer-term regime persistence and volatility dynamics. Consequently, both architectures remain widely adopted in stock movement prediction research. TODO: GRU DIAGRAM

2.2.3 Bidirectional Variants

Bidirectional recurrent neural networks (BRNNs) extend standard RNN architectures by processing sequences in both forward and backward directions (?). By combining hidden representations from past-to-future and future-to-past passes, bidirectional models leverage full-sequence context for each time step.

Bidirectional architectures have demonstrated improved performance in tasks such as speech recognition and sequence classification(?, ?), and they form the basis of widely used models such as BiLSTM and BiGRU. However, in financial time-series applications, strict temporal ordering must be preserved to avoid information leakage. While bidirectional models may be appropriate in offline feature extraction or representation learning contexts, their use in predictive trading systems requires careful consideration to ensure causal integrity. TODO: Bi DIAGRAM

2.2.4 Implications for Stock Movement Prediction

The progression from classical RNNs to gated recurrent architectures and their bidirectional extensions reflects a broader shift toward models capable of handling long-term dependencies, non-linearity, and temporal complexity. In financial markets—characterised by regime shifts, volatility clustering, and evolving dynamics—such properties are particularly valuable.

Nevertheless, while deep sequence models have demonstrated strong statistical performance relative to classical baselines, their effectiveness remains sensitive to market conditions and evaluation methodology. This reinforces the importance of coupling architectural advances with robust evaluation, calibration, and economic assessment.

2.2.5 Ordinal Regression for Financial Returns

Financial returns are inherently ordered quantities rather than purely categorical variables. Binary classification (up/down) ignores magnitude information, while regression directly models continuous returns but may be sensitive to noise. Ordinal regression provides an intermediate formulation by discretising returns into ordered buckets, preserving directional and magnitude structure while improving robustness. This formulation enables the estimation of expected return ranges rather than simple direction, offering more informative signals for downstream capital allocation. TODO: Insert Formula For Multi-Class Classification – Deep and Interpretable Regression Models for Ordinal Outcomes Lucas Kook

2.3 BERT for Natural Language Processing

The integration of textual information into stock prediction models has attracted increasing attention, as market prices often reflect narratives and sentiment not immediately captured by historical returns alone. ? demonstrated that incorporating Twitter data alongside price time-series improves predictive performance relative to price-only baselines. Early approaches typically relied on static word embeddings (e.g., Word2Vec or GloVe) combined with recurrent or convolutional encoders to extract textual features. For example, ? encoded textual posts using neural attention mechanisms alongside price features, while ? model textual relationships through graph-based structures to capture inter-stock dependencies.

These methods, however, depend on context-independent embeddings, where each word is represented by a fixed vector regardless of surrounding context. Transformer-based language models, particularly BERT ?, address this limitation by generating contextualised embeddings in which a word's representation depends on its sentence-level context. BERT employs a bidirectional self-attention mechanism, enabling it to capture long-range linguistic dependencies and nuanced semantic relationships more effectively than earlier recurrent encoders. TODO: BERT Transformer Diagram

Domain-specific adaptations such as FinBERT ? further improve performance in financial applications fine-tuning the model on financial corpora, enhancing its ability to interpret domain-specific terminology, sentiment, and stance. Empirical studies report consistent improvements across sentiment classification and financial text analysis tasks when using such pre-trained transformer models. TODO: Difference between BERT and FinBert

Despite these advances, textual signals in financial markets remain noisy, sparse, and temporally irregular. Careful alignment between text timestamps and market data is therefore essential to prevent information leakage. Furthermore, improvements in textual representation quality do not automatically translate into economic gains, reinforcing the need for integrated evaluation within realistic trading frameworks.

2.4 Probability Calibration

In classification models, outputs are often constrained to lie between zero and one, typically through sigmoid or softmax activations, and are therefore interpreted as probabilities. However, these values represent model confidence rather than true probabilistic quantities in the formal sense of Kolmogorov's axioms (?), as demonstrated by ?. A neural network does not estimate the objective probability of a stock moving up or down, rather, it produces a score reflecting patterns learned from the training data and encoded in its parameters.

Even under ideal conditions—clean data, large sample sizes, and well-optimised architectures—it is unrealistic to expect model outputs to coincide exactly with empirical outcome frequencies. Deep neural networks are known to exhibit systematic overconfidence, particularly when trained to minimise cross-entropy loss. As a result, a predicted probability of 0.7 does not necessarily imply that the event occurs approximately 70% of the time.

This distinction is particularly consequential in financial applications. Capital allocation strategies such as the Kelly Criterion and MPT explicitly require probability estimates to determine optimal position sizing. If predicted probabilities are miscalibrated, allocation decisions can systematically overestimate or underestimate risk, leading to excessive leverage, unstable trading behaviour, and increased drawdowns.

A model is therefore said to be well-calibrated when its predicted probabilities align with observed empirical frequencies. Probability calibration methods aim to correct systematic distortions in model confidence, enabling outputs to be interpreted more reliably for downstream risk-sensitive decision-making.

2.4.1 Isotonic Regression

Isotonic regression is a non-parametric post-hoc calibration technique that fits a monotonic transformation to model outputs in order to align predicted confidence scores with empirical outcome frequencies. Originally proposed for probability

estimation in supervised learning by ?, isotonic regression relaxes parametric assumptions and instead learns a piecewise-constant monotonic mapping using the pool-adjacent-violators (PAV) algorithm.

This flexibility allows isotonic regression to correct systematic distortions in model confidence while preserving ranking. Empirical studies demonstrate that isotonic calibration can significantly improve probability estimates for binary and multiclass classification tasks ??. However, because it is non-parametric and highly flexible, isotonic regression may overfit when calibration datasets are small, leading to unstable probability estimates in limited-data regimes ?. TODO: Isotonic Calibration Diagram

2.4.2 Temperature Scaling

Temperature scaling is a parametric post-hoc calibration method introduced for modern deep neural networks by ?. The method rescales the logits of a trained classifier using a single scalar temperature parameter $T > 0$, learned on a held-out validation set. The calibrated probabilities are obtained by applying the softmax function to the rescaled logits.

Unlike non-parametric methods such as isotonic regression, temperature scaling preserves the relative ranking of predictions while correcting systematic overconfidence. Empirical evidence demonstrates that deep neural networks trained with cross-entropy loss are often miscalibrated, typically exhibiting overconfident predictions, and that temperature scaling substantially reduces calibration error without affecting classification accuracy ?.

Because it introduces only a single additional parameter, temperature scaling is less prone to overfitting than flexible non-parametric approaches and performs reliably even with limited calibration data. As a result, it has become a standard baseline for post-hoc probability calibration in deep learning applications. TODO: Temp Scaling Diagram

2.5 Meta-Features and Model Self-Assessment

Financial markets are inherently non-stationary, with volatility, return dynamics, and cross-asset relationships evolving across economic cycles and structural shifts. As surveyed by ?, financial time-series exhibit high volatility, nonlinear dependencies, and regime changes associated with macroeconomic shocks, crises, and policy transitions. These characteristics imply that predictive relationships are unstable over time and that model performance may deteriorate as market conditions change.

Despite this, many predictive systems implicitly assume static reliability, treating model outputs as equally trustworthy across all market regimes. Prior forecasting errors are rarely incorporated into subsequent predictions, despite containing information about recent model performance and potential regime shifts.

? discuss the value of hybrid approaches in financial forecasting, combining complementary techniques such as evolutionary algorithms and neural networks to enhance predictive robustness in complex and potentially non-stationary market environments. Similarly, ? show that stacked deep learning architectures can achieve consistently lower out-of-sample forecasting errors, reporting reductions in MSE and MAE across heterogeneous time-series datasets, suggesting a potential pathway towards improved predictive stability under evolving data conditions.

Incorporating prior predictions and realised errors as meta-features extends this adaptive perspective by modelling time-varying predictive reliability directly within the forecasting pipeline. Rather than assuming constant model confidence, such an approach allows the system to adjust implicitly to recent performance degradation. This idea is conceptually related to meta-learning and adaptive model weighting, yet its integration within deep learning-based financial trading systems remains relatively limited.

By explicitly modelling reliability dynamics, predictive systems may improve stability and reduce overconfidence during adverse market conditions, thereby supporting more consistent downstream decision-making. TODO: Diagram of Staking?

3 Analysis/Requirements

What is the problem that you want to solve, and how did you arrive at it?

3.1 Guidance

Make it clear how you derived the constrained form of your problem via a clear and logical process.

The analysis chapter explains the process by which you arrive at a concrete design. In software engineering projects, this will include a statement of the requirement capture process and the derived requirements.

In research projects, it will involve developing a design drawing on the work established in the background, and stating how the space of possible projects was sensibly narrowed down to what you have done.

4 Design

How is this problem to be approached, without reference to specific implementation details?

4.1 Guidance

Design should cover the abstract design in such a way that someone else might be able to do what you did, but with a different language or library or tool. This might include overall system architecture diagrams, user interface designs (wireframes/personas/etc.), protocol specifications, algorithms, data set design choices, among others. Specific languages, technical choices, libraries and such like should not usually appear in the design. These are implementation details.

5 Implementation

What did you do to implement this idea, and what technical achievements did you make?

5.1 Guidance

You can't talk about everything. Cover the high level first, then cover important, relevant or impressive details.

5.2 General guidance for technical writing

These points apply to the whole dissertation, not just this chapter.

5.2.1 Figures

Always refer to figures included, like Figure ??, in the body of the text. Include full, explanatory captions and make sure the figures look good on the page. You may include multiple figures in one float, as in Figure ??, using `subcaption`, which is enabled in the template.

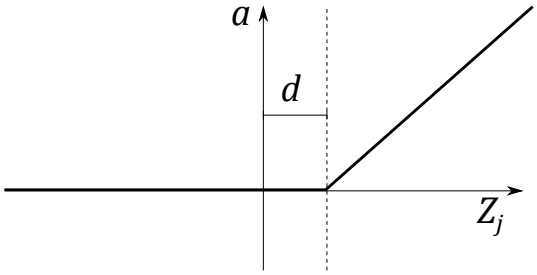
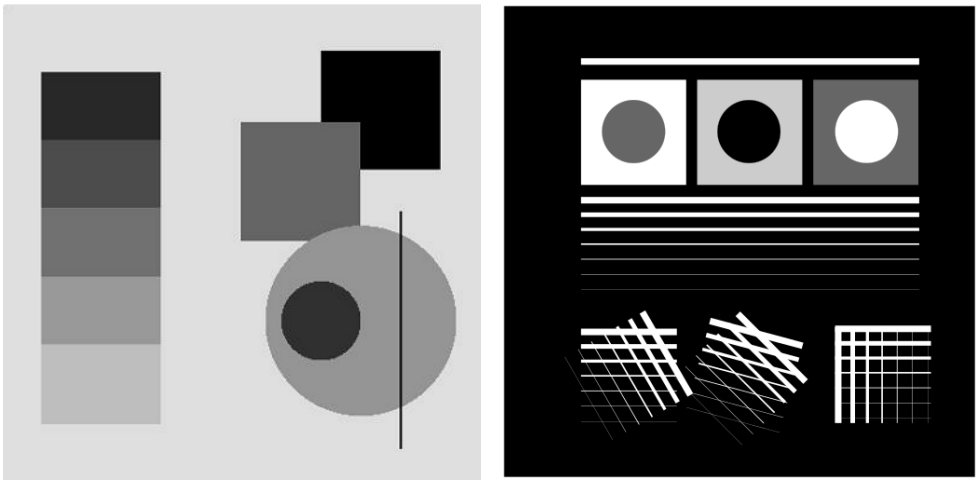


Figure 5.1: *In figure captions, explain what the reader is looking at: "A schematic of the rectifying linear unit, where* a *is the output amplitude,* d *is a configurable dead-zone, and* Z_j *is the input signal", as well as why the reader is looking at this: "It is notable that there is no activation* at all *below 0, which explains our initial results."* ***Use vector image formats (.pdf) where possible****. Size figures appropriately, and do not make them over-large or too small to read.*



(a) *Synthetic image, black on white.* ***(b)*** *Synthetic image, white on black.*

Figure 5.2: *Synthetic test images for edge detection algorithms. (??) shows various gray levels that require an adaptive algorithm. (??) shows more challenging edge detection tests that have crossing lines. Fusing these into full segments typically requires algorithms like the Hough transform. This is an example of using subfigures, with* `subref`*s in the caption.*

5.2.2 Equations

Equations should be typeset correctly and precisely. Make sure you get parenthesis sizing correct, and punctuate equations correctly (the comma is important and goes *inside* the equation block). Explain any symbols used clearly if not defined earlier.

For example, we might define:

$$\hat{f}(\xi) = \frac{1}{2}\left[\int_{-\infty}^{\infty} f(x) e^{2\pi i x \xi}\right], \tag{5.1}$$

where $\hat{f}(\xi)$ is the Fourier transform of the time domain signal $f(x)$.

5.2.3 Algorithms

Algorithms can be set using `algorithm2e`, as in Algorithm ??.

Data: $f_X(x)$, a probability density function returing the density at x.
σ a standard deviation specifying the spread of the proposal distribution.
x_0, an initial starting condition.
Result: $s = [x_1, x_2, \ldots, x_n]$, n samples approximately drawn from a distribution with PDF $f_X(x)$.
begin
    $s \longleftarrow []$
    $p \longleftarrow f_X(x)$
    $i \longleftarrow 0$
    while $i < n$ **do**
        $x' \longleftarrow \mathcal{N}(x, \sigma^2)$
        $p' \longleftarrow f_X(x')$
        $a \longleftarrow \frac{p'}{p}$
        $r \longleftarrow U(0, 1)$
        if $r < a$ **then**
            $x \longleftarrow x'$
            $p \longleftarrow f_X(x)$
            $i \longleftarrow i + 1$
            append x to s
        end
    end
end

Algorithm 1: The Metropolis-Hastings MCMC algorithm for drawing samples from arbitrary probability distributions, specialised for normal proposal distributions $q(x'|x) = \mathcal{N}(x, \sigma^2)$. The symmetry of the normal distribution means the acceptance rule takes the simplified form.

5.2.4 Tables

If you need to include tables, like Table ??, use a tool like https://www.tablesgenerator.com/ to generate the table as it is extremely tedious otherwise.

5.2.5 Code

Avoid putting large blocks of code in the report (more than a page in one block, for example). Use syntax highlighting if possible, as in Listing ??.

Table 5.1: *The standard table of operators in Python, along with their functional equivalents from the* `operator` *package. Note that table captions go above the table, not below. Do not add additional rules/lines to tables.*

Operation	**Syntax**	**Function**
Addition	`a + b`	`add(a, b)`
Concatenation	`seq1 + seq2`	`concat(seq1, seq2)`
Containment Test	`obj in seq`	`contains(seq, obj)`
Division	`a / b`	`div(a, b)`
Division	`a / b`	`truediv(a, b)`
Division	`a // b`	`floordiv(a, b)`
Bitwise And	`a & b`	`and_(a, b)`
Bitwise Exclusive Or	`a ^b`	`xor(a, b)`
Bitwise Inversion	`~a`	`invert(a)`
Bitwise Or	`a | b`	`or_(a, b)`
Exponentiation	`a ** b`	`pow(a, b)`
Identity	`a is b`	`is_(a, b)`
Identity	`a is not b`	`is_not(a, b)`
Indexed Assignment	`obj[k] = v`	`setitem(obj, k, v)`
Indexed Deletion	`del obj[k]`	`delitem(obj, k)`
Indexing	`obj[k]`	`getitem(obj, k)`
Left Shift	`a <<b`	`lshift(a, b)`
Modulo	`a % b`	`mod(a, b)`
Multiplication	`a * b`	`mul(a, b)`
Negation (Arithmetic)	`- a`	`neg(a)`
Negation (Logical)	`not a`	`not_(a)`
Positive	`+ a`	`pos(a)`
Right Shift	`a >>b`	`rshift(a, b)`
Sequence Repetition	`seq * i`	`repeat(seq, i)`
Slice Assignment	`seq[i:j] = values`	`setitem(seq, slice(i, j), values)`
Slice Deletion	`del seq[i:j]`	`delitem(seq, slice(i, j))`
Slicing	`seq[i:j]`	`getitem(seq, slice(i, j))`
String Formatting	`s % obj`	`mod(s, obj)`
Subtraction	`a - b`	`sub(a, b)`
Truth Test	`obj`	`truth(obj)`
Ordering	`a <b`	`lt(a, b)`
Ordering	`a <= b`	`le(a, b)`

```
def create_callahan_table(rule="b3s23"):
   """Generate the lookup table for the cells."""
   s_table = np.zeros((16, 16, 16, 16), dtype=np.uint8)
   birth, survive = parse_rule(rule)

   # generate all 16 bit strings
   for iv in range(65536):
      bv = [(iv >> z) & 1 for z in range(16)]
      a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p = bv

      # compute next state of the inner 2x2
      nw = apply_rule(f, a, b, c, e, g, i, j, k)
      ne = apply_rule(g, b, c, d, f, h, j, k, l)
      sw = apply_rule(j, e, f, g, i, k, m, n, o)
      se = apply_rule(k, f, g, h, j, l, n, o, p)

      # compute the index of this 4x4
      nw_code = a | (b << 1) | (e << 2) | (f << 3)
      ne_code = c | (d << 1) | (g << 2) | (h << 3)
      sw_code = i | (j << 1) | (m << 2) | (n << 3)
      se_code = k | (l << 1) | (o << 2) | (p << 3)

      # compute the state for the 2x2
      next_code = nw | (ne << 1) | (sw << 2) | (se << 3)

      # get the 4x4 index, and write into the table
      s_table[nw_code, ne_code, sw_code, se_code] = next_code

   return s_table
```

Listing 5.1: *The algorithm for packing the* 3×3 *outer-totalistic binary CA successor rule into a* $16 \times 16 \times 16 \times 16$ *4 bit lookup table, running an equivalent, notionally 16-state* 2×2 *CA.*

6 Evaluation

How good is your solution? How well did you solve the general problem, and what evidence do you have to support that?

6.1 Guidance

- Ask specific questions that address the general problem.
- Answer them with precise evidence (graphs, numbers, statistical analysis, qualitative analysis).
- Be fair and be scientific.
- The key thing is to show that you know how to evaluate your work, not that your work is the most amazing product ever.

6.2 Evidence

Make sure you present your evidence well. Use appropriate visualisations, reporting techniques and statistical analysis, as appropriate. The point is not to dump all the data you have but to present an argument well supported by evidence gathered.

If you use numerical evidence, specify reasonable numbers of significant digits; don't state "18.41141% of users were successful" if you only had 20 users. If you average *anything*, present both a measure of central tendency (e.g. mean, median) *and* a measure of spread (e.g. standard deviation, min/max, interquartile range).

You can use `siunitx` to define units, space numbers neatly, and set the precision for the whole LaTeX document.

For example, these numbers will appear with two decimal places: 3.14, 2.72, and this one will appear with reasonable spacing 1 000 000.00.

If you use statistical procedures, make sure you understand the process you are using, and that you check the required assumptions hold in your case.

If you visualise, follow the basic rules, as illustrated in Figure **??**:

- Label everything correctly (axis, title, units).
- Caption thoroughly.
- Reference in text.
- **Include appropriate display of uncertainty (e.g. error bars, Box plot)**

- Minimize clutter.

See the file `guide_to_visualising.pdf` for further information and guidance.

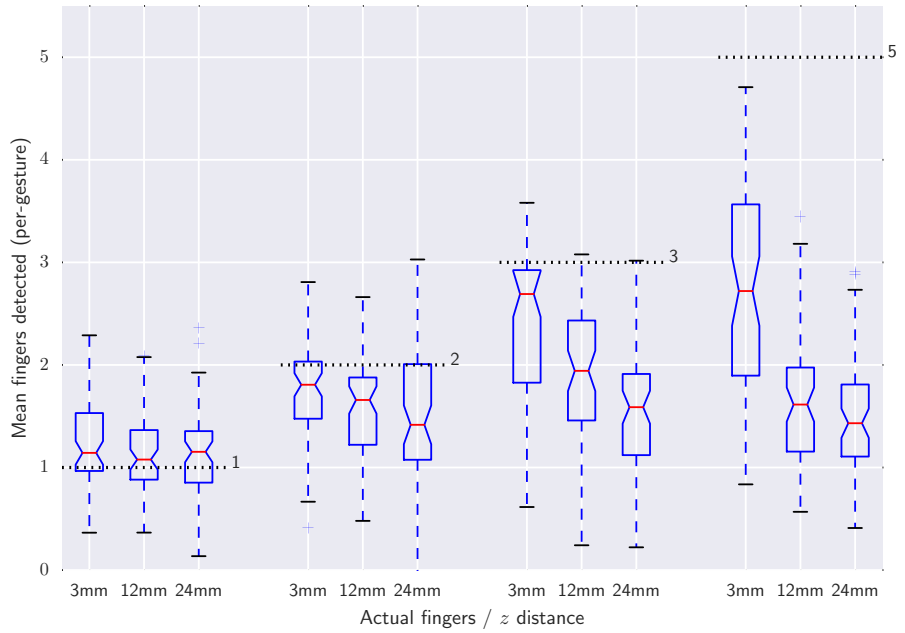


Figure 6.1: *Average number of fingers detected by the touch sensor at different heights above the surface, averaged over all gestures. Dashed lines indicate the true number of fingers present. The Box plots include bootstrapped uncertainty notches for the median. It is clear that the device is biased toward undercounting fingers, particularly at higher z distances.*

7 Conclusion

Summarise the whole project for a lazy reader who didn't read the rest (e.g. a prize-awarding committee). This chapter should be short in most dissertations; maybe one to three pages.

7.1 Guidance

- Summarise briefly and fairly.
- You should be addressing the general problem you introduced in the Introduction.
- Include summary of concrete results ("the new compiler ran 2x faster")
- Indicate what future work could be done, but remember: **you won't get credit for things you haven't done.**

7.2 Summary

Summarise what you did; answer the general questions you asked in the introduction. What did you achieve? Briefly describe what was built and summarise the evaluation results.

7.3 Reflection

Discuss what went well and what didn't and how you would do things differently if you did this project again.

7.4 Future work

Discuss what you would do if you could take this further – where would the interesting directions to go next be? (e.g. you got another year to work on it, or you started a company to work on this, or you pursued a PhD on this topic)

A Appendices

Use separate appendix chapters for groups of ancillary material that support your dissertation. Typical inclusions in the appendices are:

- Copies of ethics approvals (you must include these if you needed to get them)
- Copies of questionnaires etc. used to gather data from subjects. Don't include voluminous data logs; instead submit these electronically alongside your source code.
- Extensive tables or figures that are too bulky to fit in the main body of the report, particularly ones that are repetitive and summarised in the body.
- Outline of the source code (e.g. directory structure), or other architecture documentation like class diagrams.
- User manuals, and any guides to starting/running the software. Your equivalent of `readme.md` should be included.

Don't include your source code in the appendices. It will be submitted separately.

Bibliography

Araci, D. (2019), 'Finbert: Financial sentiment analysis with pre-trained language models'.
URL: *https://arxiv.org/abs/1908.10063*

Cahuantzi, R., Chen, X. and Güttel, S. (2023), *A Comparison of LSTM and GRU Networks for Learning Symbolic Sequences*, Springer Nature Switzerland, p. 771–785.
URL: *http://dx.doi.org/10.1007/978-3-031-37963-5*$_5$3

Devlin, J., Chang, M.-W., Lee, K. and Toutanova, K. (2019), 'Bert: Pre-training of deep bidirectional transformers for language understanding'.
URL: *https://arxiv.org/abs/1810.04805*

Feng, F., Chen, H., He, X., Ding, J., Sun, M. and Chua, T.-S. (2019), 'Enhancing stock movement prediction with adversarial training'.
URL: *https://arxiv.org/abs/1810.09936*

Fischer, T. and Krauss, C. (2018), 'Deep learning with long short-term memory networks for financial market predictions', *European Journal of Operational Research* **270**(2), 654–669.
URL: *https://www.sciencedirect.com/science/article/pii/S0377221717310652*

Graves, A. and Schmidhuber, J. (2005), 'Framewise phoneme classification with bidirectional lstm and other neural network architectures', *Neural Networks* **18**(5), 602–610. IJCNN 2005.
URL: *https://www.sciencedirect.com/science/article/pii/S0893608005001206*

Guo, C., Pleiss, G., Sun, Y. and Weinberger, K. Q. (2017), 'On calibration of modern neural networks'.
URL: *https://arxiv.org/abs/1706.04599*

He, S. and Gu, S. (2022), 'Multi-modal attention network for stock movements prediction'.
URL: *https://arxiv.org/abs/2112.13593*

Ho, M. K., Darman, H. and Musa, S. (2021), 'Stock price prediction using arima, neural network and lstm models', *Journal of Physics: Conference Series* **1988**(1), 012041.
URL: *https://doi.org/10.1088/1742-6596/1988/1/012041*

Hochreiter, S. and Schmidhuber, J. (1997), 'Long short-term memory', *Neural computation* **9**(8), 1735–1780.

Huang, Z., Xu, W. and Yu, K. (2015), 'Bidirectional lstm-crf models for sequence tagging'.
URL: *https://arxiv.org/abs/1508.01991*

Li, W., Bao, R., Harimoto, K., Chen, D., Xu, J. and Su, Q. (2020), Modeling the stock relation with graph network for overnight stock movement prediction, *in* C. Bessiere, ed., 'Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI-20', International Joint Conferences on Artificial Intelligence Organization, pp. 4541–4547. Special Track on AI in FinTech.
URL: *https://doi.org/10.24963/ijcai.2020/626*

Niculescu-Mizil, A. and Caruana, R. (2005), Predicting good probabilities with supervised learning, pp. 625–632.

Pesaran, M. H. and Timmermann, A. (2002), 'Market timing and return prediction under model instability', *Journal of Empirical Finance* **9**(5), 495–510.
URL: *https://ideas.repec.org/a/eee/empfin/v9y2002i5p495-510.html*

Schuster, M. and Paliwal, K. (1997), 'Bidirectional recurrent neural networks', *IEEE Transactions on Signal Processing* **45**(11), 2673–2681.

Schwert, G. W. (1989), 'Why does stock market volatility change over time?', *The Journal of Finance* **44**(5), 1115–1153.
URL: *https://onlinelibrary.wiley.com/doi/abs/10.1111/j.1540-6261.1989.tb02647.x*

Staudemeyer, R. C. and Morris, E. R. (2019), 'Understanding lstm – a tutorial into long short-term memory recurrent neural networks'.
URL: *https://arxiv.org/abs/1909.09586*

Varshney, R. P. and Sharma, D. K. (2024), 'Optimizing time-series forecasting using stacked deep learning framework with enhanced adaptive moment estimation and error correction', *Expert Systems with Applications* **249**, 123487.
URL: *https://www.sciencedirect.com/science/article/pii/S095741742400352X*

von Plato, J. (2005), Chapter 75 - a.n. kolmogorov, grundbegriffe der wahrscheinlichkeitsrechnung (1933), *in* I. Grattan-Guinness, R. Cooke, L. Corry, P. Crépel and N. Guicciardini, eds, 'Landmark Writings in Western Mathematics 1640-1940', Elsevier Science, Amsterdam, pp. 960–969.
URL: *https://www.sciencedirect.com/science/article/pii/B978044450871350156X*

Xu, Y. and Cohen, S. B. (2018), Stock movement prediction from tweets and historical prices, *in* I. Gurevych and Y. Miyao, eds, 'Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)', Association for Computational Linguistics, Melbourne, Australia,

pp. 1970–1979.
URL: *https://aclanthology.org/P18-1183/*

Zadrozny, B. and Elkan, C. (2001), Obtaining calibrated probability estimates from decision trees and naive bayesian classifiers, *in* 'Proceedings of the Eighteenth International Conference on Machine Learning', ICML '01, Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, p. 609–616.

Zadrozny, B. and Elkan, C. (2002), Transforming classifier scores into accurate multiclass probability estimates, *in* 'Proceedings of the Eighth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining', KDD '02, Association for Computing Machinery, New York, NY, USA, p. 694–699.
URL: *https://doi.org/10.1145/775047.775151*